Kiến trúc Microservices

Từ kiến trúc nguyên khối đến hệ thống phân tán hiện đại

Người trình bày: Sinh viên thực hiện

Thời lượng: 10 phút

1. Monolith bộc lộ giới hạn khi hệ thống mở rộng

- Toàn bộ tính năng gói trong một codebase
- Một module lỗi khiến toàn hệ thống sập
- Khó mở rộng linh hoạt từng tính năng
- Triển khai chậm do phụ thuộc chéo lớn

2. Microservices chia nhỏ ứng dụng theo miền nghiệp vụ

- Tách nhỏ thành nhiều dịch vụ độc lập
- Mỗi dịch vụ phụ trách một nghiệp vụ riêng
- Có thể dùng công nghệ và ngôn ngữ khác nhau
- Phát triển và đóng gói triển khai độc lập

3. Mỗi dịch vụ tự quản lý cơ sở dữ liệu riêng

- Áp dụng nguyên tắc "Database per Service"
- Tránh việc chia sẻ trực tiếp database chung
- Đảm bảo tính đóng gói dữ liệu tuyệt đối
- Dữ liệu trao đổi qua API có kiểm soát

4. Giao tiếp qua API Gateway và Event-Driven Messaging

- API Gateway làm điểm tiếp nhận duy nhất
- Giao tiếp đồng bộ qua RESTful API hoặc gRPC
- Giao tiếp bất đồng bộ qua Message Broker (Kafka/RabbitMQ)
- Giúp giảm độ trễ và tránh phụ thuộc chặt

5. Đẩy mạnh tốc độ phát hành và khả năng chịu lỗi

- Cho phép release tính năng liên tục hàng ngày
- Lỗi dịch vụ này không làm sập dịch vụ khác
- Tự động scale đúng thành phần quá tải
- Tối ưu chi phí hạ tầng máy chủ

6. Đánh đổi bằng độ phức tạp quản lý và dữ liệu

- Mất tính nhất quán dữ liệu tức thời (ACID)
- Phải áp dụng tính nhất quán sau cùng (Eventual Consistency)
- Khó debug và theo dõi lỗi phân tán
- Đòi hỏi năng lực DevOps và CI/CD cao

7. Microservices không phải "viên đạn bạc" cho mọi dự án

- Dự án nhỏ nên bắt đầu từ Monolith sạch
- Chuyển đổi khi hệ sinh thái đủ lớn
- Đội ngũ cần đủ nhân lực vận hành hạ tầng
- Độ phức tạp bài toán phải đủ lớn

8. Tóm tắt 3 bài học cốt lõi cần nhớ

- Microservices giải quyết bài toán quy mô và độc lập
- Đánh đổi tính tiện lợi lấy năng lực mở rộng
- Luôn cân nhắc giữa bài toán kinh doanh và chi phí

9. Cảm ơn và Hỏi đáp (Q&A)

- Tài liệu tham khảo: *Building Microservices* (Sam Newman)
- Bài viết kiến trúc: *martinfowler.com/microservices*
- Sẵn sàng tiếp nhận câu hỏi thảo luận!